A* Search Algorithm

Complete Explanation with Teacher's Notation

Contents

1 Introduction

A* (pronounced "A-star") is one of the most important and widely-used informed search algorithms in artificial intelligence. It combines the strengths of Uniform Cost Search (UCS) and Greedy Best-First Search (GBFS) to find optimal solutions efficiently.

What Makes A* Special?

A* combines:

- **Actual cost so far** $g(n)$ – like UCS (guarantees optimality)
- **Estimated cost to goal** $h(n.State)$ – like GBFS (provides guidance)
- **Total estimated cost** $f(n) = g(n) + h(n.State)$ – orders the search

1.1 Key Characteristics

- **Strategy**: Orders nodes by $f(n) = g(n) + h(n.State)$
- **Complete**: Yes (with safe heuristic)
- **Optimal**: Yes (with admissible heuristic)
- **Time Complexity**: Exponential in general, but much better than uninformed search
- **Space Complexity**: Keeps all nodes in memory

2 Key Concepts Before Starting

2.1 The Evaluation Function

Understanding f(n)

$$f(n) = g(n) + h(n.State)$$

where:

$$\begin{aligned} g(n) &= \text{actual cost from start to node } n \\ h(n.State) &= \text{estimated cost from state } n.State \text{ to goal} \\ f(n) &= \text{estimated total cost of path through node } n \end{aligned}$$

Intuition: $f(n)$ estimates the cost of the cheapest solution passing through node n.

2.2 Admissible Heuristics

Admissibility Requirement

A heuristic h is **admissible** if:

$$h(s) \leq h^*(s) \quad \text{for all states } s$$

where $h^*(s)$ is the true optimal cost from state s to the nearest goal.
Key Point: An admissible heuristic *never overestimates* the cost to reach a goal.

2.3 Consistent (Monotonic) Heuristics

Consistency Property

A heuristic h is **consistent** (or **monotonic**) if for every state s and successor state s' reached by action a with cost c:

$$h(s) \leq c + h(s')$$

Relationship: Consistency implies admissibility, but not vice versa.
Advantage: With consistent heuristics, A* never needs to reopen nodes from the closed set.

2.4 Node Structure

In the teacher's notation, each search node n has the following properties:

- $n.State$ – the state in the state space
- $n.Pred$ – pointer to predecessor node (for path reconstruction)
- $n.Action$ – the action that led to this node
- $g(n)$ – the cost of the path from the initial node to n

3 The A* Algorithm

3.1 Complete Algorithm Pseudocode (Teacher's Version)

A* Search Algorithm (Following Lecture 3

```
function A*(admissible heuristic h) returns a solution, or "unsolvable"
    n ← InitNode()
                                                ▷ Define: f(n) = g(n) + h(n.State)
    open ← Empty queue of search nodes ordered by ascending f-value
    closed ← ∅                                  ▷ Set of already processed states
    Insert n into open
    while open is not empty do
        n ← Pop(open)                           ▷ Pop search node n with lowest f(n)
        if GoalTest(n.State) then
            return ExtractSolution(n)
        end if
        closed ← closed ∪ {n.State}
        for each a ∈ Actions(n.State) do
            n' ← SuccNode(n, a)                 ▷ Recall that g(n') = g(n) + cost(a)
            if n'.State ∉ closed and n'.State ∉ States(open) then
                Insert n' into open
            else if ∃n'' ∈ open s.t. n''.State = n'.State and g(n') < g(n'') then
                Replace n'' in open with n'
            else if ∃n'' ∈ closed s.t. n''.State = n'.State and g(n') < g(n'') then
                Remove n'' from closed and insert n' into open
            end if
        end for
    end while
    return "unsolvable"
end function
```

4 Line-by-Line Explanation

4.1 Initialization Phase (Lines 1-6)

4.1.1 Line 1: Function Declaration

```
function A*(admissible heuristic h) returns a solution, or "unsolvable"
```

Purpose: Declares the A* search function that takes an admissible heuristic as a parameter.
Input:

- An admissible heuristic function h that estimates the cost from any state to a goal

Output:

- A solution (sequence of actions/states) if one exists
- The string "unsolvable" if no solution exists

4.1.2 Line 2: Initialize Root Node

```
n ← InitNode()
```

Purpose: Creates the initial search node corresponding to the start state of the problem.
Details:

- $n.State$ is set to the initial state I
- $n.Pred =$ `null` (no predecessor)
- $n.Action =$ `null` (no action led here)
- $g(n) = 0$ (no cost accumulated yet)
- $f(n) = 0 + h(I) = h(I)$

4.1.3 Line 3: Define f(n)

```
/* Define: f(n) = g(n) + h(n.State) */
```

Purpose: Reminds us how the evaluation function f is computed for any node n.
Key Point: This is the core of A* – combining known cost $g(n)$ with estimated remaining cost $h(n.State)$.

4.1.4 Line 4: Initialize Open List

```
open ← Empty queue of search nodes ordered by ascending f-value
```

Purpose: Creates the open list (frontier) as a priority queue.
Key Details:

- Implemented as a priority queue (min-heap)
- Nodes ordered by $f(n) = g(n) + h(n.State)$ (lowest first)
- Contains search *nodes*, not just states
- Will initially contain only the root node (after line 6)

Priority Queue Ordering

The priority queue ensures we always expand the node with the lowest estimated total cost $f(n)$ first. This is the core of A*'s efficiency and optimality.

4.1.5 Line 5: Initialize Closed Set

```
closed ←     /* Set of already processed states */
```

Purpose: Creates the closed set to track already-expanded states.
Important: The closed set stores *states*, not nodes. This is because:

- Multiple nodes can have the same state (different paths to same state)
- We want to avoid re-expanding the same state
- We check membership: $n.State \in closed$

4.1.6 Line 6: Insert Initial Node

```
Insert n into open
```

Purpose: Adds the initial node to the open list to start the search.
State After Initialization:

- $open = \{n_0\}$ where n_0 is the initial node with $f(n_0) = h(I)$
- $closed = \emptyset$

4.2 Main Search Loop (Lines 7-21)

4.2.1 Line 7: While Loop Condition

```
while open is not empty do
```

Purpose: Continues searching as long as there are unexplored nodes.
Termination Conditions:

- **Success**: Goal found (line 10)
- **Failure**: Open list becomes empty (line 22)

4.2.2 Line 8: Select Best Node

```
n ← Pop(open)    /* Pop search node n with lowest f(n) */
```

Purpose: Removes and returns the node with the lowest f-value from the open list.
Key Point: This is the node that A* believes is most promising – the one with lowest estimated total path cost.

Node Selection Strategy

A* always expands the node with the lowest $f(n) = g(n) + h(n.State)$ value. This combines:

- What we **know**: $g(n)$ (actual cost from start to n)
- What we **estimate**: $h(n.State)$ (estimated cost from n to goal)

This strategy ensures that A* expands nodes in order of their estimated total cost, leading to optimality when h is admissible.

4.3 Goal Test (Lines 9-11)

4.3.1 Line 9: Test for Goal

```
if GoalTest(n.State) then
```

Purpose: Checks if the current node's state satisfies the goal condition.

Timing: Note that we test when we *expand* a node (pop from open), not when we *generate* it (insert into open).

Why This Matters: This ensures that when we find a goal, it's guaranteed to be optimal (with admissible h).

4.3.2 Line 10: Return Solution

```
return ExtractSolution(n)
```

Purpose: Constructs and returns the solution path.

How ExtractSolution Works:

1. Start with the goal node n
2. Follow $n.Pred$ pointers back to the initial node
3. Collect the actions ($n.Action$) along the way
4. Reverse the sequence to get start-to-goal path

Optimality Guarantee

When A* expands a goal node with admissible heuristic, that goal is **guaranteed to be optimal**. Here's why:

1. Let n_G be the goal node we just expanded
2. We have $f(n_G) = g(n_G) + h(n_G.State) = g(n_G) + 0 = g(n_G)$
3. All nodes remaining in open have $f(n) \geq f(n_G)$
4. If a cheaper goal existed, there would be an unexpanded node on the optimal path with $f < f(n_G)$ (because h is admissible)
5. But we expanded n_G first, so no such node exists
6. Therefore, n_G is optimal

4.4 Closing the Current Node (Line 12)

4.4.1 Line 12: Add State to Closed Set

```
closed ← closed  {n.State}
```

Purpose: Marks the current state as expanded by adding it to the closed set.

Key Points:

- We add the *state* ($n.State$), not the node
- With consistent heuristics, we never need to expand this state again
- With inconsistent heuristics, we might need to reopen (see line 19)

4.5 Successor Generation (Lines 13-20)

4.5.1 Line 13: Iterate Over Actions

```
for each a  Actions(n.State) do
```

Purpose: Iterates through all applicable actions in the current state.
Details:

- ACTIONS($n.State$) returns the set of actions applicable in state $n.State$
- Each action a has an associated cost $cost(a)$

4.5.2 Line 14: Generate Successor Node

```
n' ← SuccNode(n, a)    /* Recall that g(n') = g(n) + cost(a) */
```

Purpose: Creates a successor node by applying action a to node n.
SuccNode Creates:

- $n'.State$ = result of applying action a in state $n.State$
- $n'.Pred = n$ (pointer to parent node)
- $n'.Action = a$ (the action that led here)
- $g(n') = g(n) + cost(a)$ (accumulated path cost)
- $f(n') = g(n') + h(n'.State)$ (total estimated cost)

4.6 Duplicate Detection and Cheaper Path Check (Lines 15-20)

This is the most complex part of A*, handling three cases:

4.6.1 Line 15-16: Case 1 – New State

```
if n'.State  closed and n'.State  States(open) then
    Insert n' into open
```

Condition: The successor state is completely new (never seen before).
Action: Add the successor node to the open list.
Why: This is a new state worth exploring.

Helper Function: States(open)

STATES($open$) returns the set of states corresponding to all nodes currently in the open list. This allows us to check if we've already seen a state without searching through all nodes.

4.6.2 Line 17-18: Case 2 – Cheaper Path to State in Open

```
else if n''  open s.t. n''.State = n'.State and g(n') < g(n'') then
    Replace n'' in open with n'
```

Condition:

- There exists a node n'' in open with the same state as n'
- The new path to this state is cheaper: $g(n') < g(n'')$

Action: Replace the old node n'' with the new better node n' in the priority queue.
Why: We found a better path to a state we were already planning to explore. Update it with the cheaper cost.
Effect:

- The node's g-value decreases
- The node's f-value likely decreases
- The node's priority in the queue improves
- The parent pointer ($n'.Pred$) now points to the better predecessor

4.6.3 Line 19-20: Case 3 – Cheaper Path to State in Closed (Reopening)

```
else if n''  closed s.t. n''.State = n'.State and g(n') < g(n'') then
    Remove n'' from closed and insert n' into open
```

Condition:

- There exists a node with state $n'.State$ that was already expanded (in closed)
- The new path to this state is cheaper than the previous one

Action:

1. Remove the state from closed (reopen it)
2. Insert the new cheaper node into open

Why This is Needed:

Handling Inconsistent Heuristics

This reopening mechanism is necessary when using **inconsistent heuristics**.
With Consistent Heuristics: This case never occurs because we always find the optimal path to a state on first expansion. The triangle inequality guarantees no cheaper path can be found later.
With Inconsistent Heuristics: We might initially reach a state via a suboptimal path and expand it. Later, we might discover a cheaper path to the same state. Without reopening, we'd miss the cheaper solution.
Example: Consider states $A \to B \to C$:

- Path 1: $A \to C$ with cost 10, $h(A) = 5$, so $f = 15$
- Path 2: $A \to B$ (cost 3) $\to C$ (cost 2), $h(B) = 15$, so node B has $f = 18$
- If we expand A's successor C first (lower f), then later find cheaper path via B
- We need to reopen C to get the optimal solution

4.7 No Solution Found (Line 22)

4.7.1 Line 22: Return Failure

```
return "unsolvable"
```

Purpose: Returns failure indication when the open list becomes empty without finding a goal.
Meaning: No path exists from the initial state to any goal state. The problem is unsolvable.

5 Comparison with Other Algorithms

5.1 A* vs UCS vs GBFS

Property	UCS	GBFS	A*
Orders by	$g(n)$	$h(n)$	$g(n) + h(n)$
Complete	Yes	No	Yes
Optimal	Yes	No	Yes (if h admissible)
Uses heuristic	No	Yes	Yes
Memory	$O(b^d)$	$O(b^d)$	$O(b^d)$

5.2 Why A* Combines the Best of Both

Understanding the Combination

UCS ($g(n)$ only):

- Finds optimal solutions
- But explores uniformly in all directions
- Inefficient – doesn't use problem knowledge

GBFS ($h(n)$ only):

- Efficient – goes straight toward goal
- But ignores accumulated cost
- Not optimal – can miss cheaper paths

A* ($g(n) + h(n)$):

- Finds optimal solutions (like UCS)
- Efficient with good heuristic (like GBFS)
- Best of both worlds!

6 Properties of A*

6.1 Completeness

Completeness of A*

Theorem: A* is complete if:

- The branching factor is finite
- Edge costs are bounded below by some $\epsilon > 0$
- The heuristic is safe (doesn't return ∞ for solvable states)

Intuition: These conditions prevent infinite loops and guarantee that if a solution exists, A* will eventually find it.

6.2 Optimality

Optimality Theorem

Theorem: A* is optimal if the heuristic h is admissible.
For tree search: Admissibility is sufficient.
For graph search:

- Consistency (monotonicity) guarantees optimality without reopening
- With only admissibility (inconsistent h), we must allow reopening (line 19-20)

6.2.1 Proof Sketch (Tree Search)

Why A* with Admissible h is Optimal

Setup: Suppose A* terminates by expanding goal G_2, but G_1 is a cheaper goal.
Proof by Contradiction:

1. Let n be an unexpanded node on the optimal path to G_1
2. Since h is admissible: $h(n.State) \leq h^*(n.State)$ (true cost from n to goal)
3. Therefore: $f(n) = g(n) + h(n.State) \leq g(n) + h^*(n.State)$
4. But $g(n) + h^*(n.State)$ is the optimal cost through n, which equals $g^*(G_1)$
5. So: $f(n) \leq g^*(G_1) < g^*(G_2)$
6. But A* expanded G_2 before n, so it must have $f(G_2) \leq f(n)$
7. This gives: $g^*(G_2) = f(G_2) \leq f(n) \leq g^*(G_1) < g^*(G_2)$
8. **Contradiction!**

Conclusion: A* cannot expand a suboptimal goal before expanding all nodes that could lead to a cheaper goal.

6.3 Optimally Efficient

Optimally Efficient Among Optimal Algorithms

Remarkable Property: Among all optimal algorithms using the same heuristic information, A* is optimally efficient.
Meaning: No other optimal algorithm using the same heuristic is guaranteed to expand fewer nodes than A*.
Intuition: Any algorithm that expands fewer nodes risks missing the optimal solution. A* expands exactly the nodes necessary to guarantee optimality.

6.4 Complexity

Time and Space Complexity

Time Complexity: $O(b^d)$ in the worst case, where:

- b = branching factor
- d = depth of optimal solution

Space Complexity: $O(b^d)$ – keeps all generated nodes in memory
Practical Performance: With a good heuristic, A* performs *much better* than the worst case. The effective branching factor can be close to 1 with excellent heuristics.

7 Execution Example: Romania Problem

Let's trace A* finding the path from Arad to Bucharest using straight-line distance as the heuristic.

7.1 Problem Setup

Romania Routing Problem

Goal: Find shortest path from Arad to Bucharest
States: Romanian cities
Actions: Drive between connected cities
Costs: Road distances (km)
Heuristic: Straight-line distance to Bucharest (admissible)

7.2 Initial State

Step 0: Initialize

Initial Node: n_0 with $n_0.State = Arad$, $g(n_0) = 0$, $h(Arad) = 366$
Data Structures:

open	$\{Arad_0\}$ with $f = 366$
closed	$\emptyset$

7.3 Iteration 1: Expand Arad

Iteration 1

Pop: n with $n.State = Arad$, $f(n) = 366$
Goal Test: $Arad \neq Bucharest$ – continue
Add to closed: $closed = \{Arad\}$
Generate Successors:

City (n'.State)	cost(a)	g(n')	h(n'.State)	f(n')
Sibiu	140	140	253	393
Timisoara	118	118	329	447
Zerind	75	75	374	449

All three are new states – insert all into open.
State After:

open	$\{Sibiu_{393}, Timisoara_{447}, Zerind_{449}\}$
closed	$\{Arad\}$

7.4 Iteration 2: Expand Sibiu

Iteration 2

Pop: n with $n.State = Sibiu$, $f(n) = 393$
Goal Test: $Sibiu \neq Bucharest$ – continue
Add to closed: $closed = \{Arad, Sibiu\}$
Generate Successors:

City	cost(a)	g(n')	h	f(n')	Action
Arad	140	280	366	646	Skip (in closed)
Fagaras	99	239	178	417	Insert
Oradea	151	291	380	671	Insert
Rimnicu Vilcea	80	220	193	413	Insert

State After:

open	$\{RimnicuVilcea_{413}, Fagaras_{417}, Timisoara_{447}, Zerind_{449}, Oradea_{671}\}$
closed	$\{Arad, Sibiu\}$

7.5 Iteration 3: Expand Rimnicu Vilcea

Iteration 3

Pop: n with $n.State = RimnicuVilcea$, $f(n) = 413$
Goal Test: $RimnicuVilcea \neq Bucharest$ – continue
Add to closed: $closed = \{Arad, Sibiu, RimnicuVilcea\}$
Generate Successors:

City	cost(a)	g(n')	h	f(n')	Action
Sibiu	80	300	253	553	Skip (in closed)
Pitesti	97	317	98	415	Insert
Craiova	146	366	160	526	Insert

State After:

open	$\{Pitesti_{415}, Fagaras_{417}, Timisoara_{447}, Zerind_{449}, Craiova_{526}, Oradea_{671}\}$
closed	$\{Arad, Sibiu, RimnicuVilcea\}$

7.6 Iteration 4: Expand Pitesti

Iteration 4

Pop: n with $n.State = Pitesti$, $f(n) = 415$
Goal Test: $Pitesti \neq Bucharest$ – continue
Add to closed: $closed = \{Arad, Sibiu, RimnicuVilcea, Pitesti\}$
Generate Successors:

City	cost(a)	g(n')	h	f(n')	Action
RimnicuVilcea	97	414	193	607	Skip (in closed)
Bucharest	101	418	0	418	Insert
Craiova	138	455	160	615	Update (was 526, worse)

Note: Path to Craiova via Pitesti (cost 455) is worse than via Rimnicu Vilcea (cost 366), so

we don't update.
State After:

open	$\{Fagaras_{417}, Bucharest_{418}, Timisoara_{447}, Zerind_{449}, Craiova_{526}, Oradea_{671}\}$
closed	$\{Arad, Sibiu, RimnicuVilcea, Pitesti\}$

7.7 Iteration 5: Expand Fagaras

Iteration 5

Pop: n with $n.State = Fagaras$, $f(n) = 417$
Goal Test: $Fagaras \neq Bucharest$ – continue
Add to closed: $closed = \{Arad, Sibiu, RimnicuVilcea, Pitesti, Fagaras\}$
Generate Successors:

City	cost(a)	g(n')	h	f(n')	Action
Sibiu	99	338	253	591	Skip (in closed)
Bucharest	211	450	0	450	Skip (in open, worse)

Note: Path to Bucharest via Fagaras (cost 450) is worse than via Pitesti (cost 418), so we don't update.
State After:

open	$\{Bucharest_{418}, Timisoara_{447}, Zerind_{449}, Craiova_{526}, Oradea_{671}\}$
closed	$\{Arad, Sibiu, RimnicuVilcea, Pitesti, Fagaras\}$

7.8 Iteration 6: Goal Found!

Iteration 6 – SUCCESS!

Pop: n with $n.State = Bucharest$, $f(n) = 418$
Goal Test: **YES! Bucharest is the goal!**
Return: $\textsc{ExtractSolution}(n)$
Solution Path (following $n.Pred$ pointers):

$$Arad \xrightarrow{140} Sibiu \xrightarrow{80} RimnicuVilcea \xrightarrow{97} Pitesti \xrightarrow{101} Bucharest$$

Total Cost: $g(n) = 140 + 80 + 97 + 101 = 418$ km
This is the optimal solution!

7.9 Why is This Optimal?

Optimality Verification

When A* expanded Bucharest, we had:

- $f(Bucharest) = 418$ (the goal node we expanded)
- All remaining nodes in open have $f \geq 418$:
  - $Timisoara$: $f = 447$
  - $Zerind$: $f = 449$
  - $Craiova$: $f = 526$
  - $Oradea$: $f = 671$

Since all unexpanded nodes have $f \geq 418$, and the heuristic is admissible (straight-line distance never overestimates), no cheaper path to any goal can exist.
Therefore, the solution with cost 418 is **optimal**.

8 Heuristic Design

8.1 What Makes a Good Heuristic?

Properties of Good Heuristics

A good heuristic should be:

1. **Admissible**: Never overestimate the true cost (for optimality)
2. **Consistent**: Satisfy $h(s) \leq cost(a) + h(s')$ (avoids reopening)
3. **Informative**: As close to h^* as possible (for efficiency)
4. **Efficient to compute**: Fast evaluation (low overhead)

8.2 Heuristic Dominance

Dominance

If $h_2(s) \geq h_1(s)$ for all states s, and both are admissible, then h_2 **dominates** h_1.
Result: A* with h_2 will expand fewer or equal nodes than with h_1.
Intuition: A higher (but still admissible) heuristic is more informed and guides search more effectively toward the goal.

8.3 Common Heuristics

8.3.1 8-Puzzle Heuristics

1. $h_0(s) = 0$ (uninformed – reduces to UCS)
2. $h_1(s) =$ number of misplaced tiles
3. $h_2(s) =$ sum of Manhattan distances of tiles from goal positions
4. $h_3(s) = h_2(s) + 2\times$ (number of linear conflicts)

Dominance: h_3 dominates h_2 dominates h_1 dominates h_0
Performance Impact:

- With h_1: A* expands ~500 nodes on average
- With h_2: A* expands ~50 nodes on average
- With h_3: A* expands ~20 nodes on average

8.3.2 Grid Pathfinding Heuristics

1. **Manhattan distance**: $|x_1 - x_2| + |y_1 - y_2|$ (for 4-connected grids)
2. **Euclidean distance**: $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$ (for 8-connected grids)
3. **Chebyshev distance**: $\max(|x_1 - x_2|, |y_1 - y_2|)$ (for diagonal movement)

9 Practical Considerations

9.1 When to Use A*

A* is Ideal When...

- You need an **optimal solution**
- You have a **good admissible heuristic**
- You have **sufficient memory** (A* stores all generated nodes)
- The problem has **reasonable branching factor and depth**

9.2 Alternatives and Variants

When A* is Not Suitable

Limited Memory:

- **IDA*** (Iterative Deepening A*): Uses depth-first search with f-cost bound
- **SMA*** (Simplified Memory-Bounded A*): Drops worst nodes when memory full
- **RBFS** (Recursive Best-First Search): Uses linear space

Suboptimal Solutions Acceptable:

- **Weighted A***: Use $f(n) = g(n) + W \cdot h(n)$ with $W > 1$
- Finds solutions up to W times optimal, but much faster
- **Greedy Best-First Search**: Use only $h(n)$, no optimality guarantee

Bidirectional Search Possible:

- **Bidirectional A***: Search from both start and goal simultaneously
- Can significantly reduce search space

9.3 Common Pitfalls

9.3.1 Pitfall 1: Non-Admissible Heuristic

Problem and Solution

Problem: Using a heuristic that overestimates can cause A* to return suboptimal solutions.
Example: Using straight-line distance when there are obstacles that force detours longer than the straight line.
Solution: Always verify admissibility. Ensure $h(s) \leq h^*(s)$ for all reachable states s.

9.3.2 Pitfall 2: Forgetting to Handle Duplicates

Importance of Duplicate Detection

Without proper duplicate detection (lines 15-20), A* would:

- Generate the same state multiple times
- Waste enormous amounts of time and memory
- Potentially never terminate in graphs with cycles

The cheaper path check (lines 17-20) is essential for both correctness and efficiency.

9.3.3 Pitfall 3: Inconsistent Heuristic Without Reopening

Problem and Solution

Problem: With inconsistent heuristics, closing nodes permanently (without reopening) can cause A* to miss the optimal solution.
Solution: Either:

1. Use a consistent heuristic (preferred), OR
2. Implement the reopening logic (lines 19-20)

Most practical heuristics are consistent, so reopening is rarely needed in practice.

10 Summary

A* Search – Key Takeaways

1. **Best-first search with** $f(n) = g(n) + h(n.State)$: Combines actual cost and estimated remaining cost
2. **Optimal and complete**: With admissible heuristic and standard completeness conditions
3. **Optimally efficient**: No other optimal algorithm using same heuristic can do better
4. **Admissibility ensures optimality**: Never overestimating allows safe pruning
5. **Consistency avoids reopening**: Triangle inequality property prevents need for line 19-20
6. **Heuristic quality is critical**: Better heuristics lead to exponentially faster search
7. **Memory intensive**: Stores all generated nodes – main practical limitation
8. **Widely applicable**: Pathfinding, planning, puzzle solving, game AI, robotics, and more

Algorithm	Orders by	Complete	Optimal	Space
BFS	Depth	Yes	Yes*	$O(b^d)$
UCS	$g(n)$	Yes	Yes	$O(b^d)$
GBFS	$h(n)$	No	No	$O(b^d)$
yellow1 **A***	$g(n) + h(n)$	Yes	Yes**	$O(b^d)$

*BFS optimal only with unit costs
**A* optimal when h is admissible